

LABORATORY OBSERVATION

Course	Full Stack Web Development
Course Code	23CM4121
Experiment No.	6
Student Name	Kavya G.
Roll No	A24126552082

1. TITLE

NPM and package.json – Installing and Using an External Package in Node.js

2. AIM

To understand NPM and package.json, and to demonstrate the steps involved in installing and using an external package in a Node.js application.

3. OBJECTIVE

- To understand the purpose of NPM in Node.js.
 - To understand the role of package.json and dependencies.
 - To learn the commands used to initialize a project and install a package.
 - To use the Express package to create a simple web server.
-

4. THEORY

NPM (Node Package Manager) is a tool used to install, manage, and share packages for Node.js applications. Packages are reusable libraries that provide ready-made functionality.

package.json is a project file that stores information such as the project name, version, scripts, and dependencies. When an external package is installed, it is listed under dependencies in package.json. The installed package files are stored in the node_modules folder, while package-lock.json records the exact dependency versions.

In this experiment, Express is used as an external package. Express helps create routes and handle HTTP requests with less code.

5. ALGORITHM / PROCEDURE

1. Install Node.js, which includes NPM.
 2. Create a folder for the Node.js project and open it in the terminal.
 3. Run npm init -y to create the package.json file.
 4. Run npm install express to install Express as a project dependency.
 5. Create a file named app.js.
 6. Import Express and create an application instance.
 7. Define a GET route for the home page that sends a response.
 8. Start the server on port 3000 and run the application using node app.js.
 9. Open http://localhost:3000 in a browser to view the response.
-

6. NPM COMMANDS

Command	Purpose
npm init -y	Creates package.json with default settings.
npm install express	Installs Express and adds it to dependencies.
node app.js	Runs the Node.js application.

7. PROGRAM

```
const express = require('express');

const app = express();

app.get('/', (req, res) => {
  res.send('Hello Node.js');
});

app.listen(3000, () => {
  console.log('Server running on port 3000');
});
```

8. CODE EXPLANATION

Code / Function	Explanation
require("express")	Imports the installed Express package.
express()	Creates an Express application.
app.get('/', ...)	Defines a GET route for the home page (/).
res.send(...)	Sends "Hello Node.js" as the response.
app.listen(3000, ...)	Starts the server on port 3000.
console.log(...)	Displays a message when the server starts.

9. OUTPUT

```
● PS D:\kavya\Tutorials\FS lab\Observation_Tasks\task 6\middleware> npm.cmd init -y
Wrote to D:\kavya\Tutorials\FS lab\Observation_Tasks\task 6\middleware\package.json:

{
  "name": "middleware",
  "version": "1.0.0",
  "description": "",
  "main": "code.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "dependencies": {
    "express": "^5.2.1"
  }
}

● PS D:\kavya\Tutorials\FS lab\Observation_Tasks\task 6\middleware> npm.cmd install express

added 68 packages, and audited 69 packages in 1s

28 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
○ PS D:\kavya\Tutorials\FS lab\Observation_Tasks\task 6\middleware> node code.js
Server running on port 3000
█
```

10. RESULT

Thus, NPM and package.json were studied successfully. The Express package was installed and used to create a Node.js web server that responds with “Hello Node.js”.